COMP4121 Project 6 Report: Polymorphic Types

MAKSIMOVICH, Roman
rmaksimovich@connect.ust.hk

WU, Yiu Tsz
ytwuac@connect.ust.hk

1. Introduction

In its existing implementation, the Amy language lacks two prominent features of functional languages: lambda abstractions and parametric polymorphism. We have implemented the latter, i.e. supporting

- polymorphic types of the form `T[A_1, ... A_n]`, where the type variables `A_k` may be instantiated to concrete types and constructors extending `T` may depend on `A_k` in the types of their arguments;
- polymorphic functions of the form `f[A_1, ... A_n] (...)`, whose argument types and return type may depend on `A_1, ... A_n`.

The syntax is the same as outlined in the proposal, namely

```
abstract class T[A_1, A_2, ..., A_k]   // type T parameterized by variables `A_j`
case class C(a_1: R_1, ..., a_m: R_m) extends T
def fun[B_1, ..., B_n, C] (a_1: T_1, ..., a_l: T_l): S := ... end fun
```

Here the types `R_i` may contain any of the type variables `A_1, ..., A_k`, and the types `T_1, ..., T_l, S` may contain any of the variables `B_1, ..., B_n`.

In addition, we have undertaken and completed the extra challenge of rewriting the entire compiler frontend from scratch in Rust, including

1. A naïve implementation of the lexer, without reliance on regular expressions;
2. A recursive-descent implementation of the parser;
3. A custom implementation of the name analyzer;
4. A custom implementation of the type-checker, with support for polymorphism;
5. A custom implementation of the interpreter.

Due to time constraints, we have chosen to *skip the code generation stage* and only implement the interpreter. We have also chosen to implement *equality by value* for strings and constructors, instead of equality by reference, both because it is simpler and because it is more useful.

Our projects provides, apart from the extra language features, a state-of-the-art error report system, emitting messages that look like this:

Input:

```
1  object Test
2    abstract class Option[A]
3
4    case class None() extends Option
5    case class Some(x: A) extends W
6  end Test
```

Output:

```
Error during the resolving stage.
-> ./Test.amy:5:14

4       case class None() extends Option
5       case class !! Some(x: A) extends W !!
6     end Test

Could not find a type named `W` in the
`Test` module.
```

2. Examples

Consider the following module which defines a `List[A]` type:

```
object TL
  abstract class List[A]
  case class Nil() extends List
  case class Cons(h: A, t: List[A]) extends List

  def switchType[A,B] (xs: List[A]): List[B] :=
    xs match {
      case Nil() => Nil()
      case Cons(_, _) => error("no way to produce B from A.")
    }
  end switchType

  def length[C] (lst: List[C]): Int(32) :=
    lst match {
      case Nil() => 0
      case Cons(h, t) => 1 + length(t)
    }
  end length

  val xs: List[Int(32)] = Cons(3, Cons(7, Nil()));
  val ys: List[Boolean] = switchType(xs);
  ys
end TL
```

This program type-checks and the type variable `A` in `List[A]` is specified to different types in different scenarios. The following program, on the other hand, will not type-check:

```
object Test
  abstract class Triple[A, B, C]
  case class MkTriple(x: A, y: B, z: C) extends Triple

  def getFirst[X, Y, Z] (t: Triple[X, Y, Z]): X :=
    t match {
      case MkTriple(x, y, z) => y
    }
  end getFirst
end Test
```

This is because the variable `y` was given type `Y` which is incompatible with the result type `X`.

3. Implementation

3.1. Lexer ans parser

In our implementation, lexing and parsing are combined into a single stage.

An input file is lexed by means of a `TokenIter<'a>` struct, which holds a reference `src: &'a [u8]` to the bytes of the file, and a moving index `position: usize`. The next token is produced by analyzing the array `src` starting from the index `position`, producing a `Token` instance, and then moving `position` forward. This implementation is powerful because it allows for cheap lookahead, which simply consists of indexing into `src`.

Our parser does not directly make use of context-free grammars. Instead, it is implemented using a set of mutually recursive functions, each responsible for parsing a particular language structure.

The public interface consists of the function

```
pub fn parse<'a>(src: &'a [u8], ts: &mut TokenIter) -> Result<NominalModule, Report>
```

which receives a pointer to an input file and a token iterator, producing a `NominalModule`.

3.2. Name analyzer

Name analysis consists of assigning unique labels to symbols throughout the program, and also checking for scope correctness, definition uniqueness, etc. In our implementation, it is done by the function

```
pub fn resolve(
    modules: VecDeque<NominalModule>,
    sg: &mut SymbolGenerator,
) -> Result<SymbolicProgram, Report>
```

which receives a list of `NominalModule`'s and a `SymbolGenerator` (which can provide fresh symbols), producing a `SymbolicProgram`. A noticeable difference from the reference compiler is that a list of nominal modules is always merged into a *single* symbolic program. This is done for simplicity and convenience, since after resolving module names no longer matter.

3.3. Type-checker

Our type-checker is very similar to the implemented in the labs, but the introduction of polymorphism requires some changes. Namely, we split type variables into two kinds: *rigid* and *fluid*. Rigid type variables are visible to the user, they occur as type parameters in type and function definitions:

```
abstract class T[A, B, C]  // A, B, C are rigid type variables
def id[A] (x: A): A := x end id  // A is rigid
```

In constraints, rigid variables act as *concrete types,* i.e. a constraint `A = A` will be solved, but `A = Int(32)` will not.
On the other hand, *fluid* type variables have the same role as before: a constraint of the form `a = T`, where `a` is a fluid variable and `T` is any type, will result in `a` being substituted with `T` in all subsequent constraints.
With regard to constraint accumulation, our implementation mostly follows the standard unification algorithm, with the exception of constructor/function calls.
Suppose we are typing a function call `f(e_1, e_2, e_3)` with expected type `T`. From the symbol table we obtain the definition of `f`:

```
def f[A,B] (x_1: P, x_2: Q, x_3: R): S := <body> end f
```

where the types `P`, `Q`, `R`, `S` may contain the type variables `A` and `B`. Now, we cannot simply generate the constraint `T = S`, since the variables `A` and `B` are rigid, and this constraint will likely not be solved. Instead, we generate fresh *fluid* variables `a`, `b` and perform an α-conversion `A -> a`, `B -> b` on the types `P`, `Q`, `R`, and `S`. We then proceed to type the call `f(e_1, e_2, e_3)` as usual.

Since for different calls of `f`, different fluid variables will be generated, we see, for example, that both `id(false)` and `id(5)` are well-typed. Constructor calls are handled similarly.

The public interface of the type-checker is the function

```
pub fn typecheck(
  program: &SymbolicProgram,
  sg: &mut SymbolGenerator
) -> Result<(), Report>
```

3.4. Interpreter

The interpreter is also mostly similar to how it is implemented in the labs. It is implemented by interpreting all expressions in a `SymbolicProgram` which contains expressions and relevant definitions obtained from parsing, name analysis and type checking. Each expression is evaluated recursively until one obtains an atomic `Value`, or if an error has occurred.

A deviation from the expected Amy semantics is taken here as a feature, of which value equality is used for strings and case class values.

The public interface of the interpreter is the function

```
pub fn interpret_program(program: SymbolicProgram) -> Result<(), Report>
```

3.5. Error handling

As you might have noticed, the `parse`, `resolve`, `typecheck`, and `interpret` functions return a result type. Our implementation is completely exception-free, and all errors are encoded in a special `Report` struct which, upon generation, is propagated to the top of the execution chain, at which point it is pretty-printed.

4. Building and testing

Assuming `rustc >= 1.94` is installed, our project may be built with `cargo build` in the root directory, and ran with `cargo run -- --interpret <fname_1>.amy ... <fname_k>.amy`. See `cargo run -- --help` for more information.
The `extension-examples` directory contains test files. Files prefixed by `Error` contain type errors. Feel free to tinker with any of the files and monitor the interpreter output.

5. Possible extensions

First of all, our extension would couple very well with the introduction of function types and lambda abstractions. Apart from that, one may consider fully supporting *type constructors* of kind `* -> *` instead of only concrete types of kind `*`. Our extension is a step in this direction, since in the type `T[A]`, `T` intuitively has kind `* -> *`. Unfortunately, in our implementation currently all type constructors must be fully applied.